پاسخ تکلیف اول

سید احمد نوربخش 40232773

سوال اول

1. **کاهش افزونگی و ناسازگاری داده‌ها (Data Redundancy and Inconsistency):** در سیستم فایل، داده‌ها در فرمت‌های مختلف و در فایل‌های متعددی ذخیره می‌شوند که باعث تکرار اطلاعات در جاهای مختلف و در نتیجه بروز ناسازگاری (مثلاً تغییر آدرس مشتری در یک فایل و عدم تغییر آن در فایل دیگر) می‌شود.
2. **رفع دشواری در دسترسی به داده‌ها (Difficulty in Accessing Data):** در سیستم پردازش فایل، برای هر نیاز اطلاعاتی جدید که از قبل پیش‌بینی نشده باشد، باید یک برنامه کاربردی جدید نوشته شود که کار دشواری است.
3. **جلوگیری از انزوای داده‌ها (Data Isolation):** از آنجا که داده‌ها در فایل‌های پراکنده و با فرمت‌های گوناگون ذخیره شده‌اند، نوشتن برنامه‌های جدید برای بازیابی داده‌های مرتبط بسیار دشوار می‌شود.
4. **رفع مشکلات امنیتی (Security Problems):** در سیستم‌های فایل، دشوار است که به کاربران مختلف فقط به بخشی از داده‌ها (و نه همه آن‌ها) اجازه دسترسی داده شود.

سوال دوم:

1. **سطح فیزیکی (Physical Level):** پایین‌ترین سطح انتزاع است که چگونگی ذخیره‌سازی واقعی رکوردها (مانند رکورد استاد یا مشتری) و ساختارهای داده پیچیده سطح پایین را روی حافظه توصیف می‌کند.
2. **سطح منطقی (Logical Level):** سطح میانی است که مشخص می‌کند چه داده‌هایی در پایگاه داده ذخیره شده‌اند و چه روابطی میان آن‌ها وجود دارد (تعریف جداول، صفات و نوع داده‌ها مانند ساختار جدول instructor با فیلدهای ID، name، dept_name و salary).
3. **سطح دید (View Level):** بالاترین سطح انتزاع است که تنها بخشی از کل پایگاه داده را نمایش می‌دهد. برنامه‌های کاربردی جزئیات نوع داده‌ها را پنهان می‌کنند و همچنین می‌توان به دلایل امنیتی، اطلاعات حساسی مانند میزان حقوق کارمندان را از دید برخی کاربران مخفی نگه داشت.

سوال سوم

- **سناریو اول (تلاش هم‌زمان دو مسافر برای رزرو تنها صندلی خالی پرواز):**
  - **ویژگی مرتبط: انزوا یا جداسازی (Isolation)** (و در کنار آن حفظ **سازگاری / Consistency**).
  - **توضیح:** ویژگی Isolation تضمین می‌کند که اجرای هم‌زمان تراکنش‌ها با یکدیگر تداخل نداشته باشد و تراکنش‌های موازی به صورت ترتیبی و مستقل به نظر برسند تا از بروز خطای Race Condition و فروش یک صندلی به دو نفر جلوگیری شود.
- **سناریو دوم (پرداخت هزینه بلیط توسط مسافر اما عدم اختصاص صندلی به دلیل بروز خطا در سیستم):**
  - **ویژگی مرتبط: اتمی بودن (Atomicity)**.
  - **توضیح:** همان‌طور که در فصل اول اسلایدها (*Atomicity of updates*) اشاره شده است، یک تراکنش باید به صورت «همه یا هیچ» اجرا شود؛ یعنی یا هر دو عملیات کسر وجه و اختصاص صندلی با موفقیت کامل می‌شوند و یا در صورت بروز خطا، کل تراکنش به حالت اول بازگردانده (Rollback) شده و پول به حساب مسافر برمی‌گردد.
- **سناریو سوم (آغاز فرایند رزرو توسط یک مسافر تا مرحله پرداخت، و نهایی شدن رزرو همان صندلی توسط مسافر دیگر در همان حین):**
  - **ویژگی مرتبط: انزوا یا جداسازی (Isolation)**.
  - **توضیح:** این مشکل به دلیل عدم ایزولاسیون مناسب و قفل‌گذاری نکردن منبع در طول تراکنش رخ داده است. سیستم مدیریت تراکنش با کنترل هم‌زمانی (Concurrency Control) تحت ویژگی Isolation باید مانع از آن شود که تراکنش دوم وضعیت داده‌ای را که در حال تغییر توسط تراکنش اول است، تغییر دهد یا رزرو را نهایی کند.

سوال چهارم

۱. جدول مشتریان (Customer)

- **CustID (کلید اصلی - Primary Key):** شناسه یکتا برای هر مشتری جهت متمایزسازی رکوردها.
- **NationalCode:** کد ملی ده رقمی مشتری برای احراز هویت قانونی.
- **FullName:** نام و نام خانوادگی مشتری.

- **Phone:** شماره تماس مشتری جهت اطلاع‌رسانی و پیامک‌های بانکی.
- **City:** شهر محل سکونت مشتری.

۲. جدول شعب بانک (Branch)

- **BranchID (کلید اصلی - Primary Key):** کد یکتای هر شعبه بانک.
- **BranchName:** نام شعبه.
- **BranchCity:** شهری که شعبه در آن واقع شده است.
- **Assets:** میزان کل دارایی‌ها و بودجه شعبه.

۳. جدول حساب‌های بانکی (Account)

- **AccountNo (کلید اصلی - Primary Key):** شماره حساب یکتای بانکی.
- **CustID (کلید خارجی - Foreign Key به جدول Customer):** مشخص می‌کند که این حساب متعلق به کدام مشتری است.
- **BranchID (کلید خارجی - Foreign Key به جدول Branch):** مشخص می‌کند که حساب در کدام شعبه افتتاح شده است.
- **Balance:** موجودی فعلی حساب به ریال.
- **AccountType:** نوع حساب (مثلاً قرض‌الحسنه، جاری یا سپرده کوتاه‌مدت).

۴. جدول تراکنش‌ها (Transaction)

- **TransID (کلید اصلی - Primary Key):** کد رهگیری یکتای هر تراکنش بانکی.
- **AccountNo (کلید خارجی - Foreign Key به جدول Account):** شماره حسابی که تراکنش روی آن صورت گرفته است.
- **Amount:** مبلغ تراکنش.
- **TransDate:** تاریخ و زمان دقیق انجام تراکنش.
- **TransType:** نوع تراکنش (واریز یا برداشت).

سوال پنجم

جداول مسئله:

- Car (CarID, Model, Brand, BranchID, DailyRate)
- Customer (CustID, Name, LicenseNo, City)
- Branch (BranchID, BranchName, Location)
- Reservation (ResID, CustID, CarID, StartDate, EndDate, IsReturned)
- Payment (PayID, ResID, Amount, PayDate, Method)

بخش ۱: نام (برند) و مدل خودروهایی که توسط مشتری‌های شیرازی رزرو شده‌اند اما هنوز بازگردانده نشده‌اند

Π Brand, Model (σ City = 'Shiraz' ∧ IsReturned = false (Customer ⋈ Reservation ⋈ Car))

بخش ۲: نام مشتریانی که بیشتر از یک خودرو از برند «Toyota» رزرو کرده‌اند

ابتدا تمام رزروهای خودروهای برند تویوتا را به دست می‌آوریم و سپس با تغییر نام (Rename)، مشتریانی را می‌یابیم که حداقل دو خودروی متمایز (CarID مختلف) از این برند رزرو کرده‌اند:

ToyotaRes ← Π CustID, CarID (Reservation ⋈ (σ Brand = 'Toyota' (Car)))

MultiToyotaCust ← Π R1.CustID (σ R1.CustID = R2.CustID ∧ R1.CarID ≠ R2.CarID (ρ R1 (ToyotaRes) × ρ R2 (ToyotaRes)))

Result ← Π Name (Customer ⋈ MultiToyotaCust)

عبارت یک‌خطی معادل:

Π Customer.Name (Customer ⋈ (Π R1.CustID (σ R1.CustID = R2.CustID ∧ R1.CarID ≠ R2.CarID ∧ C1.Brand = 'Toyota' ∧ C2.Brand = 'Toyota' (ρ R1(Reservation) ⋈_{R1.CarID = C1.CarID} ρ C1(Car) × ρ R2(Reservation) ⋈_{R2.CarID = C2.CarID} ρ C2(Car)))))

بخش ۳: نام شعبه‌هایی که خودروهای آن دست‌کم یک بار رزرو شده باشند ولی برای رزروهای آن هیچ پرداختی ثبت نشده باشد

تعبیر اول (شعبه‌ای که حداقل یک رزرو بدون پرداخت دارد):

UnpaidRes ← Π ResID, CarID (Reservation) − Π ResID, CarID (Reservation ⋈ Payment)

Result ← Π BranchName (Branch ⋈ Car ⋈ UnpaidRes)

تعبیر دوم (شعبه‌ای که خودروهایش رزرو شده‌اند اما برای هیچ‌یک از رزروهای خودروهای آن شعبه اصلاً پرداختی ثبت نشده است):

BranchesWithRes ← Π BranchID (Car ⋈ Reservation)

BranchesWithPaidRes ← Π BranchID (Car ⋈ Reservation ⋈ Payment)

Result ← Π BranchName (Branch ⋈ (BranchesWithRes − BranchesWithPaidRes))

بخش ۴: نام (برند) و مدل خودروهایی که هیچ‌گاه توسط مشتریانی از همان شهری که شعبه خودرو در آن قرار دارد رزرو نشده‌اند

ابتدا خودروهایی را پیدا می‌کنیم که حداقل یک بار توسط مشتریِ هم‌شهر با شعبه خودرو رزرو شده‌اند، و سپس آن‌ها را از کل خودروها کم می‌کنیم:

SameCityCars ← Π CarID (σ Customer.City = Branch.Location (Customer ⋈ Reservation ⋈ Car ⋈ Branch))

Result ← Π Brand, Model (Car ⋈ (Π CarID (Car) − SameCityCars))

سوال ششم

جداول مسئله:

- Person (ID, name, age, gender)
- Visits (personID, agency)
- Books (personID, trip)
- Offers (agency, trip, price)

مورد ۱: تمام آژانس‌های مسافرتی که حداقل یک مشتری بالای ۸۵ سال دارند

- **جبر رابطه‌ای:**

> Π agency (Visit

s ⋈_{personID = ID} (σ age > 85 (Person)))

- **پرسش SQL:**

```
SELECT DISTINCT V.agency
FROM Visits V, Person P
WHERE V.personID = P.ID AND P.age > 85;
```

مورد ۲: تمام آژانس‌هایی که حداقل یک سفر با قیمت کمتر از ۲۵ میلیون تومان ارائه می‌کنند و فردی به نام «عموحسن» آن سفر را رزرو کرده است

- **جبر رابطه‌ای:**

> Π agency ((σ price < 25000000 (Offers)) ⋈ Book

s ⋈_{personID = ID} (σ name = 'عموحسن' (Person)))

- **پرسش SQL:**

```
SELECT DISTINCT O.agency
FROM Offers O, Books B, Person P
WHERE O.trip = B.trip
AND B.personID = P.ID
AND O.price < 25000000
AND P.name = 'عموحسن';
```

مورد ۳: تمام آژانس‌هایی که یا فقط مشتریان مرد دارند یا فقط مشتریان زن

- **جبر رابطه‌ای:**

> MaleAgencies ← Π agency (Visit

s ⋈_{personID = ID} (σ gender = 'male' (Person)))

FemaleAgencies ← Π agency (Visits ⋈_{personID = ID} (σ gender = 'female' (Person)))

Result ← (MaleAgencies − FemaleAgencies) ∪ (FemaleAgencies − MaleAgencies)

- **پرسش SQL:**

(SELECT V.agency FROM Visits V, Person P WHERE V.personID = P.ID AND P.gender = 'male'

EXCEPT

SELECT V.agency FROM Visits V, Person P WHERE V.personID = P.ID AND P.gender = 'female')

UNION

(SELECT V.agency FROM Visits V, Person P WHERE V.personID = P.ID AND P.gender = 'female'

EXCEPT

SELECT V.agency FROM Visits V, Person P WHERE V.personID = P.ID AND P.gender = 'male');

مورد ۴: افرادی که به تمام آژانس‌هایی مراجعه می‌کنند که دست‌کم یکی از سفرهایی را که آن افراد رزرو کرده‌اند، ارائه می‌دهند

برای هر فرد، مجموعه آژانس‌هایی که حداقل یکی از سفرهای رزروشده او را ارائه می‌دهند برابر است با: Π personID, agency (Books ⋈ Offers). اگر از این مجموعه، آژانس‌هایی را که فرد به آن‌ها مراجعه کرده (Visits) کم کنیم، افرادی که در تفاضل باقی می‌مانند کسانی هستند که به حداقل یکی از آن آژانس‌ها مراجعه نکرده‌اند:

- جبر رابطه‌ای:

> RequiredVisits ← Π personID, agency (Books ⋈ Offers)
>
> ViolatingPersons ← Π personI

D (RequiredVisits − Visits)

Result ← Π ID, name (Person) − Π ID, name (Person ⋈_{ID = personID} ViolatingPersons)

(یا برای همه افراد شامل کسانی که رزروی ندارند: Π ID (Person) − ViolatingPersons)

- **پرسش SQL (با استفاده از الگوی NOT EXISTS و EXCEPT در فصل سوم اسلایدها):**

SELECT P.ID, P.name

FROM Person P

WHERE NOT EXISTS (

(SELECT O.agency

FROM Books B, Offers O

WHERE B.trip = O.trip AND B.personID = P.ID)

EXCEPT

(SELECT V.agency

FROM Visits V

WHERE V.personID = P.ID)

);

مورد ۵: آژانس یا آژانس‌های مسافرتی که ارزان‌ترین سفر شهر را ارائه می‌دهند

- **جبر رابطه‌ای:**

ابتد

ا آژانس‌ها و سفرهایی را که قیمتشان از حداقل یک سفر دیگر بیشتر است پیدا کرده و از کل پیشنهادها کم می‌کنیم:

NotCheapest ← Π O1.agency, O1.trip, O1.price (σ O1.price > O2.price (ρ O1 (Offers) × ρ O2 (Offers)))

CheapestOffers ← Offers − NotCheapest

Result ← Π agency (CheapestOffers)

- **پرسش SQL:**

```
SELECT DISTINCT agency
FROM Offers
WHERE price <= ALL (SELECT price FROM Offers);
```

سوال هفتم

رابطه‌های استاندارد مسئله:

- Supplier (sid, sname, address)
- Part (pid, pname, color)
- Catalog (sid, pid, cost)

۱. عبارت اول

Π sname ((σ color = 'red' (Part)) ⋈ (σ cost < 100 (Catalog)) ⋈ Supplier)

- **معنی عبارت:** نام (sname) تمام تامین‌کنندگانی را برمی‌گرداند که حداقل یک قطعه **قرمز** رنگ را با **قیمت کمتر از ۱۰۰** عرضه می‌کنند.

۲. عبارت دوم

Π sname ((Π sid ((σ color = 'red' (Part)) ⋈ (σ cost < 100 (Catalog)))) ⋈ Supplier)

- **معنی عبارت:** این عبارت نیز دقیقاً نام (sname) تمام تامین‌کنندگانی را برمی‌گرداند که حداقل یک قطعه **قرمز رنگ** را با **قیمت کمتر از ۱۰۰** عرضه می‌کنند.
- **تفاوت مورد ۱ و ۲ در چیست؟**
  - **از نظر خروجی و معنایی (هم‌ارزی منطقی):** هر دو عبارت **کاملاً معادل (Equivalent)** هستند و روی هر نمونه از پایگاه داده، خروجی یکسانی تولید می‌کنند (مطابق مفهوم *Equivalent Queries* در انتهای فصل دوم اسلایدها). دلیل آن این است که پیوند طبیعی بین Catalog و Supplier فقط بر اساس صفت مشترک sid انجام می‌شود و سایر صفات جدول Part و Catalog در نمایش نهایی (Π sname) نقشی ندارند.
  - **از نظر بهینه‌سازی و نحوه اجرا (Performance / Query Optimization):** عبارت دوم **بهینه‌تر** است؛ زیرا پیش از انجام عمل پیوند (Join) با جدول Supplier، ابتدا با عملگر پرتو (Π sid) ستون‌های اضافی (pid, pname, color, cost) حذف می‌شوند و همچنین

سطرهای تکراری sid (تامین‌کننده‌ای که چندین قطعه قرمز زیر ۱۰۰ تومان دارد) از بین می‌روند. در نتیجه اندازه رابطه میانی بسیار کوچک‌تر شده و پیوند نهایی با جدول Supplier با هزینه و حافظه بسیار کمتری انجام می‌شود.

۳. عبارت سوم

Π sname ((σ color = 'red' (Part)) ⋈ (σ cost < 100 (Catalog)) ⋈ Supplier) ∩ Π sname ((σ color = 'green' (Part)) ⋈ (σ cost < 100 (Catalog)) ⋈ Supplier)

- **معنی عبارت:** این عبارت اشتراک (∩) دو مجموعه از نام تامین‌کنندگان را می‌گیرد. در واقع **نام (sname) هایی را برمی‌گرداند که متعلق به تامین‌کنندگانی با عرضه قطعه قرمز زیر ۱۰۰ واحد و همچنین تامین‌کنندگانی با عرضه قطعه سبز زیر ۱۰۰ واحد است.**
- **نکته ظریف فنی:** از آنجا که عملگر پرتو (Π sname) قبل از عملگر اشتراک (∩) اعمال شده است، اشتراک بر روی **نام تامین‌کنندگان (sname)** گرفته می‌شود نه شناسه آن‌ها (sid). بنابراین اگر دو تامین‌کننده مختلف با sid متفاوت وجود داشته باشند که هر دو نام یکسانی (مثلاً نام مشابه X) داشته باشند و اولی فقط قطعه قرمز زیر ۱۰۰ و دومی فقط قطعه سبز زیر ۱۰۰ عرضه کند، نام X در خروجی این عبارت ظاهر خواهد شد! (اگر هدف پیدا کردن تامین‌کننده واحدی باشد که هم قطعه قرمز زیر ۱۰۰ و هم قطعه سبز زیر ۱۰۰ عرضه می‌کند، باید اشتراک روی sid گرفته شود و در انتها Π sname اعمال گردد).